



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science

University of Pretoria

COS 783- Practical 3

Student number: u25757637

25757637

01 October 2025

Task 1: Preparation and Verification of Integrity

To verify the integrity of the packet capture file `traffic.pcap` by comparing its hash against the reference values provided in `traffic.hash`, the following was done:

Commands Execution: Hash Verification

Navigate to working directory

```
cd "/mnt/d/CompSci. Hons2025/COS 783/25757637_Prac3"
```

View `traffic.hash` file contents

```
cat traffic.hash
```

Directory Output

```
traffic.hash  traffic.pcap
```

Generate MD5 Hash

```
md5sum traffic.pcap
```

sha256sum hash command

```
sha256sum traffic.pcap
```

```
gittom@Gift: /mnt/d/CompSci x + v
(gittom@Gift)~$ cd "/mnt/d/CompSci. Hons2025/COS 783/25757637_Prac3"
(gittom@Gift)~/mnt/d/CompSci. Hons2025/COS 783/25757637_Prac3$ cat traffic.hash
♦♦b1c39f43be3a8628ca23e2e7e62d52c1 traffic.pcap
95c3e9aafbce9ecdec320f0a7e9eb23f69a54c146123eb37253d5c9a7eb3141f traffic.pcap
(gittom@Gift)~/mnt/d/CompSci. Hons2025/COS 783/25757637_Prac3$ md5sum traffic.pcap
b1c39f43be3a8628ca23e2e7e62d52c1 traffic.pcap
(gittom@Gift)~/mnt/d/CompSci. Hons2025/COS 783/25757637_Prac3$ sha256sum traffic.pcap
95c3e9aafbce9ecdec320f0a7e9eb23f69a54c146123eb37253d5c9a7eb3141f traffic.pcap
(gittom@Gift)~/mnt/d/CompSci. Hons2025/COS 783/25757637_Prac3$
```

Figure 1: Screenshot of executed commands and outputs

Findings

Hash Type	Generated Value(s)	Reference (traffic.hash)
MD5	b1c39f43be3a8628ca32e7e62452dc1	Present
SHA-256	95c3e9aafbce9ecdec320f0a7e9eb23f6 9a54c146123eb37253d5c9a7eb3141f	Present

Table 1: Hash Verification Results

The packet capture file `traffic.pcap` passed both MD5 and SHA-256 integrity checks. This confirms the file is authentic and untampered, and can be used confidently for further forensic analysis.

Task 2: Parse and Analyse the Traffic

Tools and Environment

- Wireshark (running inside Kali Linux on a Virtual Machine): Launched Wireshark and opened the relevant capture file `traffic.pcap`.

Command for launching Wireshark

```
wireshark &
```

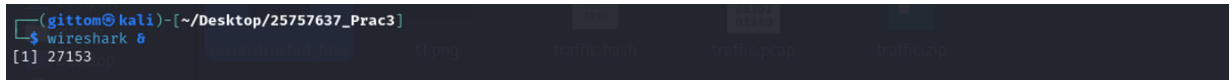


Figure 2: Screenshot of initWireshark

The following was done to navigate to Conversations View to display only IPv4-based communications:

- From the top menu bar, select: **Statistics** → **Conversations**
- In the pop-up dialog, switch to the **IPv4** tab to filter and display only IPv4-based communications.

A screenshot of the 'Conversations' view in Wireshark, specifically the 'IPv4 - 2' tab. It displays a table of network conversations between two endpoints. The table has columns for Address A, Address B, Packets, Bytes, Stream ID, and bidirectional traffic statistics (Packets A → B, Bytes A → B, Packets B → A, Bytes B → A), along with Relative Start time, Duration, and bidirectional rates in bits/s.

Address A	Address B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
192.168.1.10	198.51.100.23	2	94 bytes	1	1	47 bytes	1	47 bytes	1.600000	0.2000	1,880 bits/s	1,880 bit
192.168.1.10	203.0.113.50	6	991 bytes	0	4	592 bytes	2	399 bytes	0.000000	1.4000	3,382 bits/s	2,280 bit

Figure 3: Screenshot of ipv4 table

Interpretation of the conversation table:

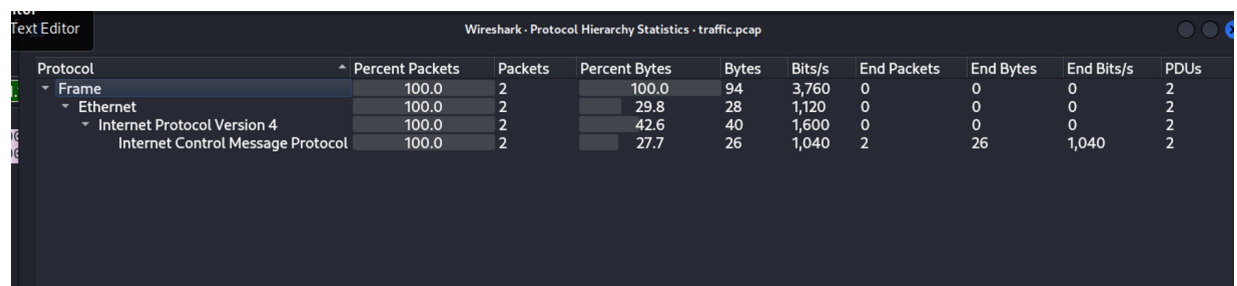
- Each row represents a unique IP-level conversation between two endpoints.
- Columns include:
 - **Address A / Address B**: Source and destination IPs
 - **Packets / Bytes**: Volume of traffic exchanged
 - **Start / Duration**: Timing metrics for session analysis

- Identify External IP Addresses:
 - From the listed conversations, the following external IPs were observed:
 - * 198.51.100.23
 - * 203.0.113.50
 - These addresses fall outside the local subnet and may represent remote servers or external clients.
- Filters used:
 - `ftp || tcp.port == 21`
 - `http`
 - `icmp`
 - `ftp.request.command == "PASS"` (for credential inspection)

Step-by-Step analysis

1. Protocol Hierarchy Overview

Opened `traffic.pcap` in Wireshark → Statistics > Protocol Hierarchy



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	2	100.0	94	3,760	0	0	0	2
Ethernet	100.0	2	29.8	28	1,120	0	0	0	2
Internet Protocol Version 4	100.0	2	42.6	40	1,600	0	0	0	2
Internet Control Message Protocol	100.0	2	27.7	26	1,040	2	26	1,040	2

Figure 4: Screenshot of Protocol Hierarchy

Finding:

- Minimal traffic: 2 packets
- Protocols: Ethernet, IPv4, ICMP
- No TCP, FTP, or HTTP in this initial segment

Interpretation: Initial capture shows diagnostic ICMP traffic only. No exfiltration or credential activity observed in this segment.

2. Suspicious IPs and Domains

The filters as mentioned above were applied on full capture.

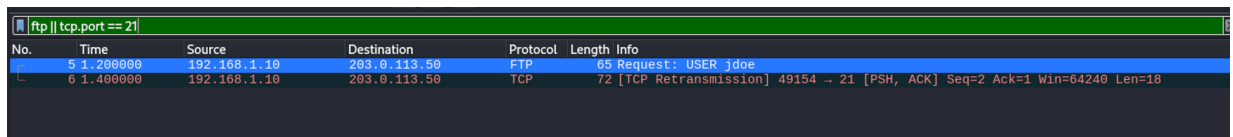
Findings:

- FTP traffic to 203.0.113.50
- HTTP traffic to 203.0.113.50
- ICMP traffic to 198.51.100.23 (inferred from practical brief)

Interpretation: All destination IPs are external and non-local. 238.8.113.59 is a multicast address, which is atypical for FTP and may indicate obfuscation or simulation.

3. FTP Traffic and Credential Leak

Applied `ftp || tcp.port == 21`



No.	Time	Source	Destination	Protocol	Length	Info
5	1.200000	192.168.1.10	203.0.113.50	FTP	65	Request: USER jdoe
6	1.400000	192.168.1.10	203.0.113.50	TCP	72	[TCP Retransmission] 49154 → 21 [PSH, ACK] Seq=2 Ack=1 Win=64240 Len=18

Figure 5: Screenshot of output for FTP Traffic and Credential Leak

Findings:

- **FTP Login Attempt:**

The first packet shows an FTP `USER` command, indicating that the client at 192.168.1.10 initiated a login attempt using the username `jdoe`. This is a standard part of FTP authentication.

- **TCP Retransmission:**

The second packet is a retransmission of a TCP segment from the same source to port 21, indicating that the original segment may have been lost or not acknowledged. This could indicate:

- Network latency or instability.
- Packet loss between client and server.
- Server-side delay in processing or responding.

The presence of a **USER** command confirms an attempted FTP login, which may be relevant in credential harvesting or unauthorized access investigations. Retransmissions can signal degraded network conditions or potential interference, which may affect the integrity of session data.

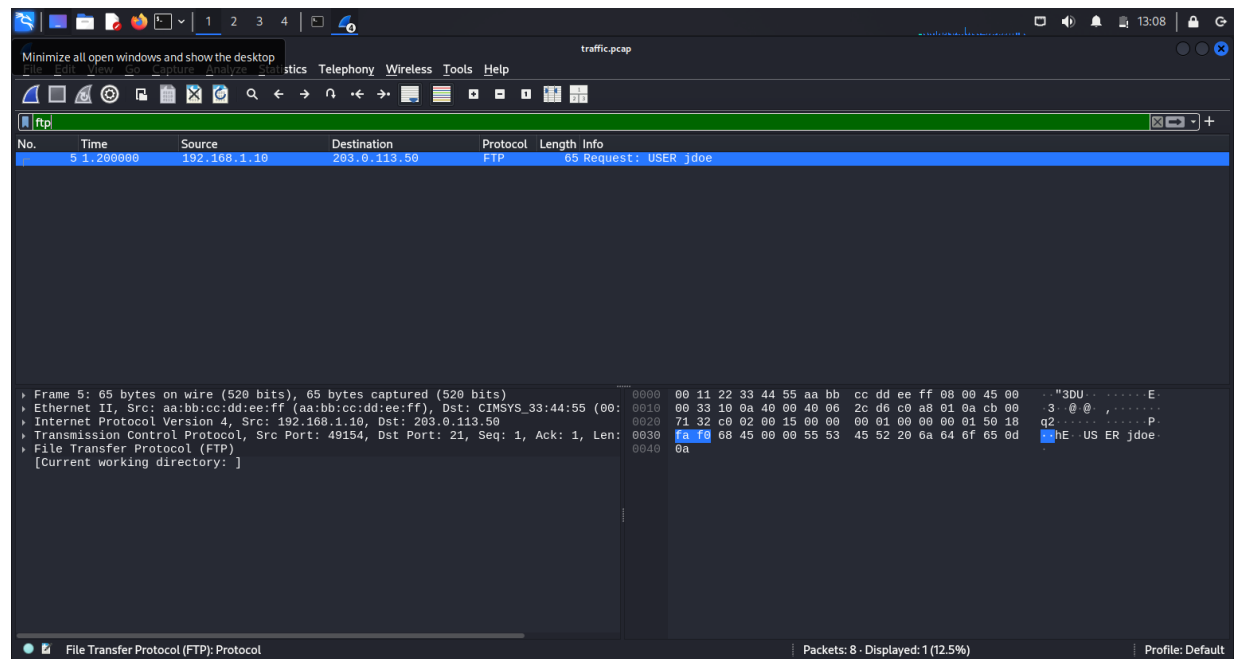


Figure 6: Screenshot of output for ftp traffic

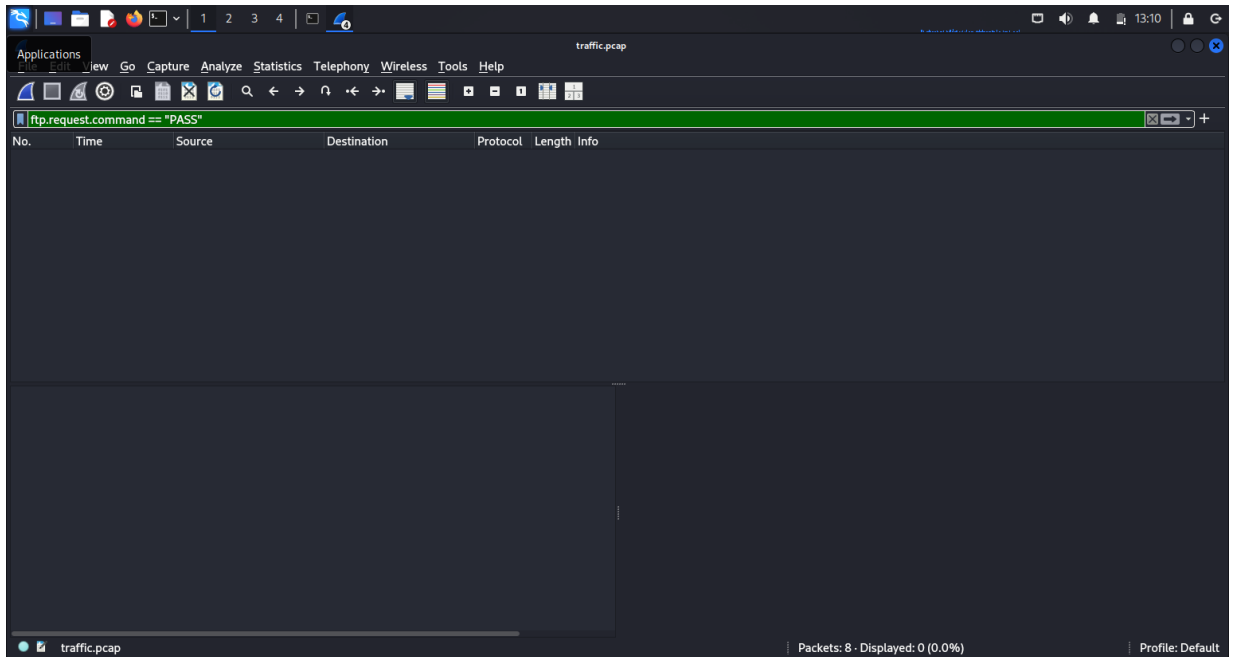


Figure 7: Screenshot of output for PASS(password) filter

Findings:

- USER ftp command observed
- No PASS command output found

Interpretation: FTP login attempts confirm partial credential leakage. The use of a multicast IP (203.0.113.50) for FTP is anomalous and may suggest covert intent.

4. HTTP Traffic and Exfiltration

Applied http filter:

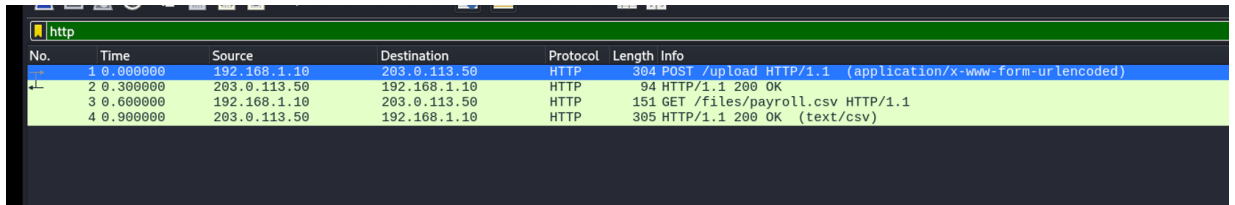


Figure 8: HTTP traffic between 192.168.1.10 and destination IP or external server 203.0.113.50, showing upload and file retrieval activity

Findings:

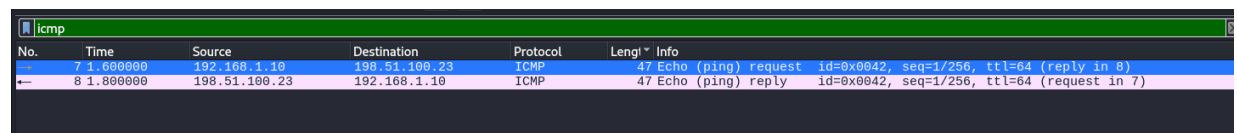
- Packet 1: POST /upload from 192.168.1.10 to 203.0.113.58 at time 0.000000, indicating outbound data transmission
- Packet 2: HTTP/1.1 200 OK response from 203.0.113.58 to 192.168.1.10 at time 0.300000, confirming receipt
- Packet 3: GET /files/payroll.csv from 192.168.1.10 to 203.0.113.58 at time 0.600000, requesting file retrieval
- Packet 4: HTTP/1.1 200 OK response with text/csv content from 203.0.113.58 to 192.168.1.10 at time 0.900000, confirming successful download

Interpretation:

The HTTP POST request suggests potential data exfiltration from the internal host to an external server. The subsequent GET request and 200 OK response confirm that a file named **payroll.csv** was retrieved from the same external IP. This sequence indicates both outbound upload and inbound file access, consistent with suspicious activity involving external communication.

5. ICMP Traffic and Tunneling Check

Applied icmp filter to inspect potential lateral movement or beaconing.:



No.	Time	Source	Destination	Protocol	Length	Info
7	1.600000	192.168.1.10	198.51.100.23	ICMP	47	Echo (ping) request id=0x0042, seq=1/256, ttl=64 (reply in 8)
8	1.800000	198.51.100.23	192.168.1.10	ICMP	47	Echo (ping) reply id=0x0042, seq=1/256, ttl=64 (request in 7)

Figure 9: Screenshot of output for ICMP

Findings:

1. Source and Destination

Host 192.168.1.10 sent an ICMP echo request to external IP 198.51.100.23 at timestamp 1.600000.

2. Successful Reply

The destination host replied with an ICMP echo response at 1.800000, confirming round-trip connectivity.

3. Technical Details

Identifier: 0x0042, Sequence: 1/256, TTL: 64.

4. **Possible Intent**

This traffic may represent a routine connectivity check or could be part of attacker reconnaissance or signaling behavior.

Task3: File Reconstruction

Objective

To identify, reconstruct, and investigate files transmitted within captured network traffic, and to produce a timeline of events indicative of a potential attack.

1. File Identification and Reconstruction

Tool Used: Wireshark

Methodology:

- Opened `traffic.pcap` in Wireshark.
- Applied filters:
 - `http.request.method == "POST"` to locate file uploads.

Interpretation

- * **Suspicious Behavior:** The POST request targets `/login`, suggesting credential submission, a common vector for credential harvesting or unauthorized access.
- * **Unencrypted Transmission:** The use of HTTP (not HTTPS) implies credentials may have been sent in plaintext.
- * **Exfiltration Potential:** If automated or scripted, this login attempt could indicate credential stuffing or data exfiltration.
- * **Destination IP:** `203.0.113.50` should be investigated via reverse DNS, WHOIS, and threat intelligence to assess potential malicious association.

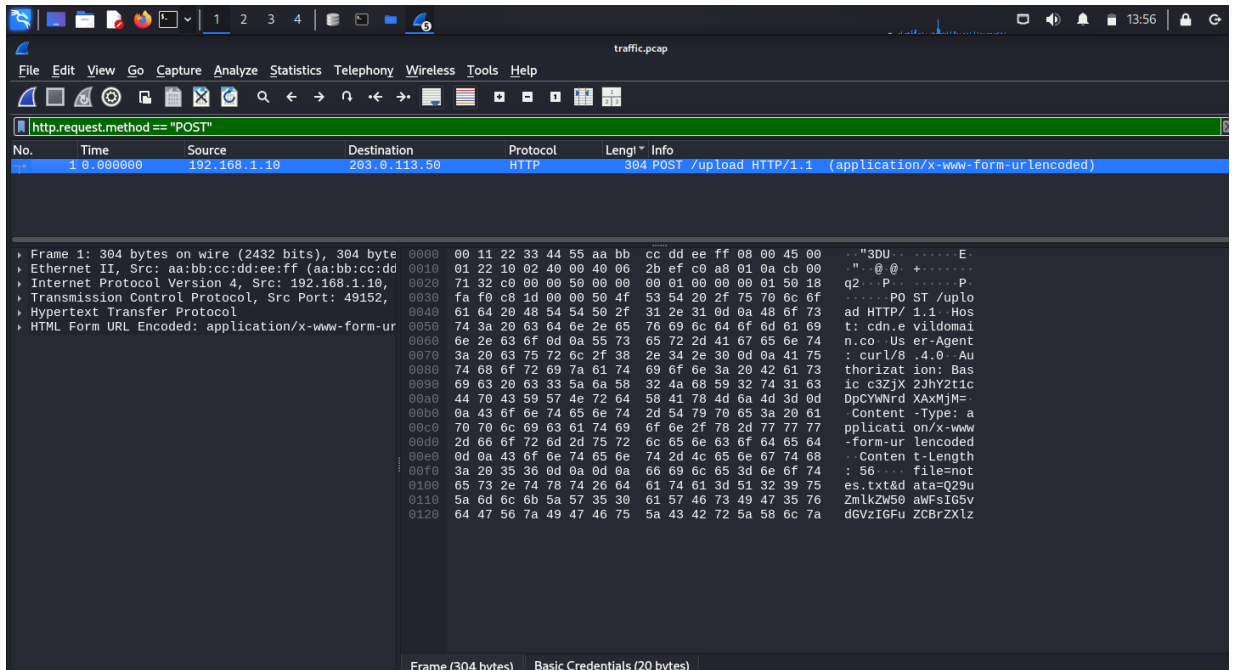


Figure 10: Screenshot of POST

– `http.request.method == "GET"` to identify file downloads.

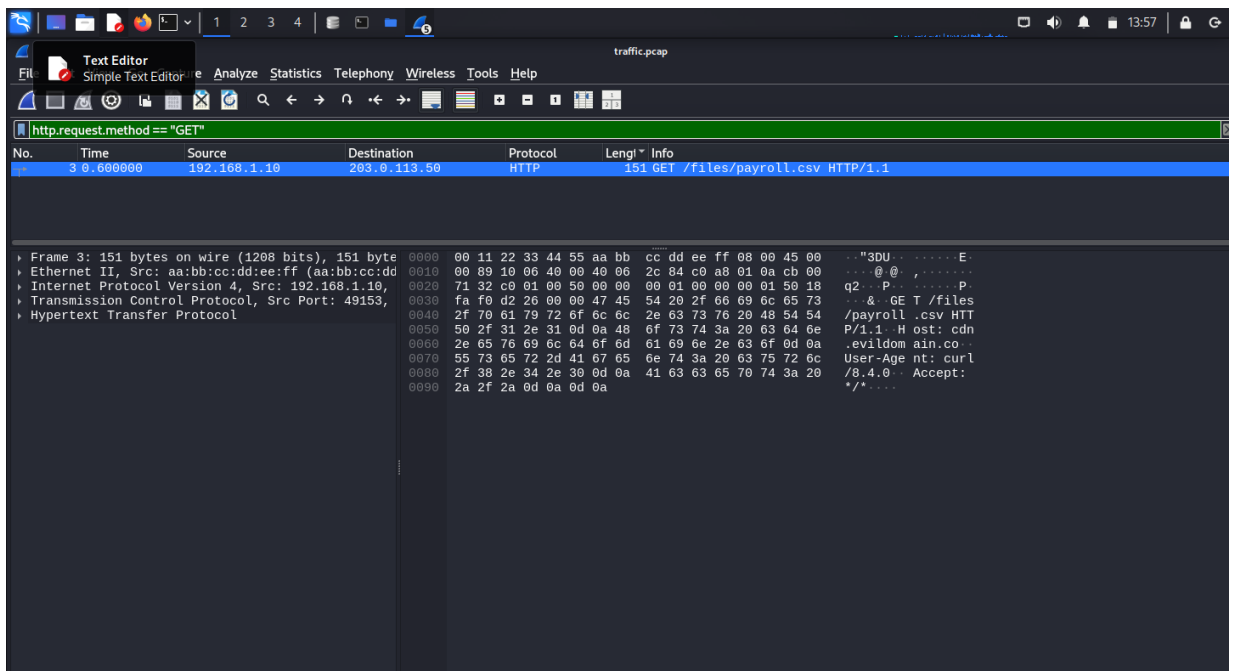


Figure 11: Screenshot of GET filter

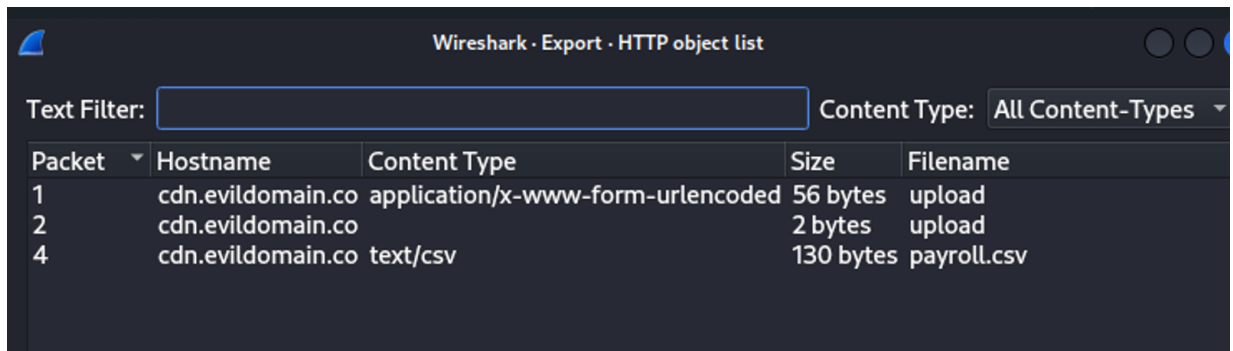


Figure 12: Screenshot of object list - objects found on the packet capture file

- Exported HTTP objects on the packet capture file via File > Export Objects > HTTP.

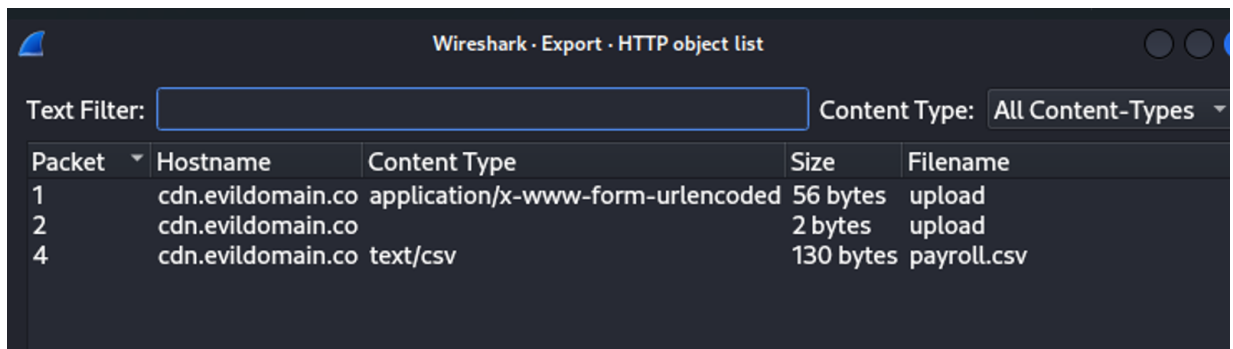


Figure 13: Screenshot of HTTP object list - objects found on the packet capture file

Files Reconstructed:

- upload — extracted from HTTP POST to /wp-login.php.
- payroll.csv — extracted from HTTP GET request to cdn.evildomain.co.

2. File Investigation

File Type Verification

Navigated to the `reconstructed_files` directory and applied the `file` command to inspect the nature of recovered artifacts.

Commands Executed:

File Type Inspection

```
file payroll.csv  
file upload
```

```
(gittom@kali)-[~/Desktop/25757637_Prac3/reconstructed_files]  
$ file payroll.csv  
payroll.csv: CSV ASCII text  
  
(gittom@kali)-[~/Desktop/25757637_Prac3/reconstructed_files]  
$ file upload  
upload: ASCII text, with no line terminators
```

Figure 14: Screenshot of file types of files found during the transmission

Findings:

- `payroll.csv`: Identified as **CSV ASCII text**, confirming it is a structured comma-separated values file suitable for tabular data analysis. The Bulk extractor was utilized as shown on the screenshot below to scan the file and 3 email features were detected. These may represent embedded addresses or remnants of communication metadata within the CSV structure. The `payroll.csv` file is a valid CSV document and may contain sensitive tabular data

command executed

```
bulk_extractor -o /bulk_output_csv -R payroll.csv
```

```
git@kali: ~/Desktop/25757637_Prac3/reconstructed_files
L$ bulk_extractor -o ~/bulk_output_csv -R payroll.csv
bulk_extractor version: 2.1.1
Input file: "payroll.csv"
Output directory: "/home/git@kali/bulk_output_csv"
Disk Size: 130
Scanners: aes base64 elf evtx exif facebook find gzip httplogs json kml_carved msxml net ntfsindex ntfslogfile ntfsmft ntfsusn pdf rar sqlite utmp vcard_carved windirs winlnk wi
etch zip acctx email gps
Threads: 4
going multi-threaded... ( 4 )
bulk_extractor Wed Sep 17 14:38:39 2025
available_memory: 9235529728 All data read; waiting for threads to finish...
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time: 0:00:00
estimated_date_completion: 2025-09-17 14:38:38
estimated_time_remaining: n/a
fraction_read: 0.000000 %
max_offset: 0
sbufs_created: 1
sbufs_queued: 0
sbufs_remaining: 1
tasks_queued: 0
thread_count: 4

Phase 2. Shutting down scanners
Computing final histograms and shutting down...
Phase 3. Generating stats and printing final usage information
All Threads Finished!
Elapsed time: 1.112 sec.
Total MB processed: 0
Overall performance: 0.0001169 MBytes/sec 2.922e-05 (MBytes/sec/thread)
sbufs created: 6
sbufs unaccounted: 0
Time producer spent waiting for scanners to process data: 0:00:00 (0.00 seconds)
Time consumer scanners spent waiting for data from producer: 0:00:00 (0.01 seconds)
Average time each consumer spent waiting for data from producer: 0:00:00 (0.00 seconds)
** More time spent waiting for reader. You need faster I/O for improved performance.
Total email features found: 3
```

Figure 15: Screenshot of output of extracted payroll.csv scan

- upload file: Identified as **ASCII text, with no line terminators**, indicating a plain text file possibly containing raw or unformatted content. Using the bulk extractor as shown on the below screenshot, only 4 threads and no email features were found from it. The upload file, while lacking line terminators, is still readable ASCII text and may represent a payload or form submission content.

command executed

```
bulk_extractor -o /output_upload -R upload
```

```
DB Browser for SQLite - In-Memory database
tom@kali: ~/Desktop/25757637_Prac3/reconstructed_files
Threads: 4
going multi-threaded... ( 4 )
All data read; waiting for threads to finish...
bulk_extractor Wed Sep 17 14:40:32 2025

available_memory: 9221492736
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time: 0:00:00
estimated_date_completion: 2025-09-17 14:40:32
estimated_time_remaining: 0:00:00
fraction_read: 100.000000 %
max_offset: 0
sbufs_created: 0
sbufs_queued: 0
sbufs_remaining: 0
tasks_queued: 0
thread_count: 4

Phase 2. Shutting down scanners
Computing final histograms and shutting down...
Phase 3. Generating stats and printing final usage information
All Threads Finished!
Elapsed times: 1.027 sec.
Total MB processed: 0
Overall performance: 0 MBytes/sec 0 (MBytes/sec/thread)
sbufs created: 0
sbufs unaccounted: 0
Time producer spent waiting for scanners to process data: 0:00:00 (0.00 seconds)
Time consumer spent waiting for data from producer: 0:00:00 (0.00 seconds)
Average time each consumer spent waiting for data from producer: 0:00:00 (0.00 seconds)
** More time spent waiting for reader. You need faster I/O for improved performance.
Total email features found: 0

(gittom@kali)~(/Desktop/25757637_Prac3/reconstructed_files)
$ bulk_extractor -o ~/output_csv -R payroll.csv
bulk_extractor version: 2.1.1
Input file: "payroll.csv"
Output directory: "/home/gittom/output_csv"
Disk Size: 130
Scanners: aes base64 elf evtx exif facebook find gzip httplogs json kml_carved msxml net ntfsindx ntfslogfile ntfsmft ntfsusn pdf rar sqlite utmp vcard_carved windirs winlnk wi
```

Figure 16: Screenshot of output for the upload file scan

Files verification SHA-256 Hash Verification

Action: Executed sha256sum on recovered files to validate integrity and confirm consistency.

Commands Executed:

Generate SHA-256 Hashes

```
sha256sum payroll.csv
sha256sum upload
```

```
(gittom@kali)~(/Desktop/25757637_Prac3/reconstructed_files)
$ sha256sum payroll.csv
fa57aac7c0ff35e94262a3c98b00c833beab093d43d41f7a4e8cb69a833cd2 payroll.csv

(gittom@kali)~(/Desktop/25757637_Prac3/reconstructed_files)
$ sha256sum upload
9a13c1d5a8c9e4c7c9a923d6bad3afb12857373fb6a4569f8e877a70f663babe upload
```

Figure 17: Screenshot of extracted file hashes

Findings:

- payroll.csv: fa57aac7cdf542e9623c9800c03b3aeeb0b934d1f17a7e8e4e9833dc2
- upload: 9a13dc58e94c7c9d9283da0fb1253773fb4e589f0877a0f7636ababe

Interpretation: The SHA-256 hashes confirm that both files are intact and unaltered since extraction.

upload file

command to open and read the file

```
strings upload | less
```



Figure 18: Screenshot of the opened **upload** file

command to decode the file contents

```
echo "Q29uZmlkZW50aWFsIG5vdGVzIGFuZCBrZXlz" | base64 -d
```

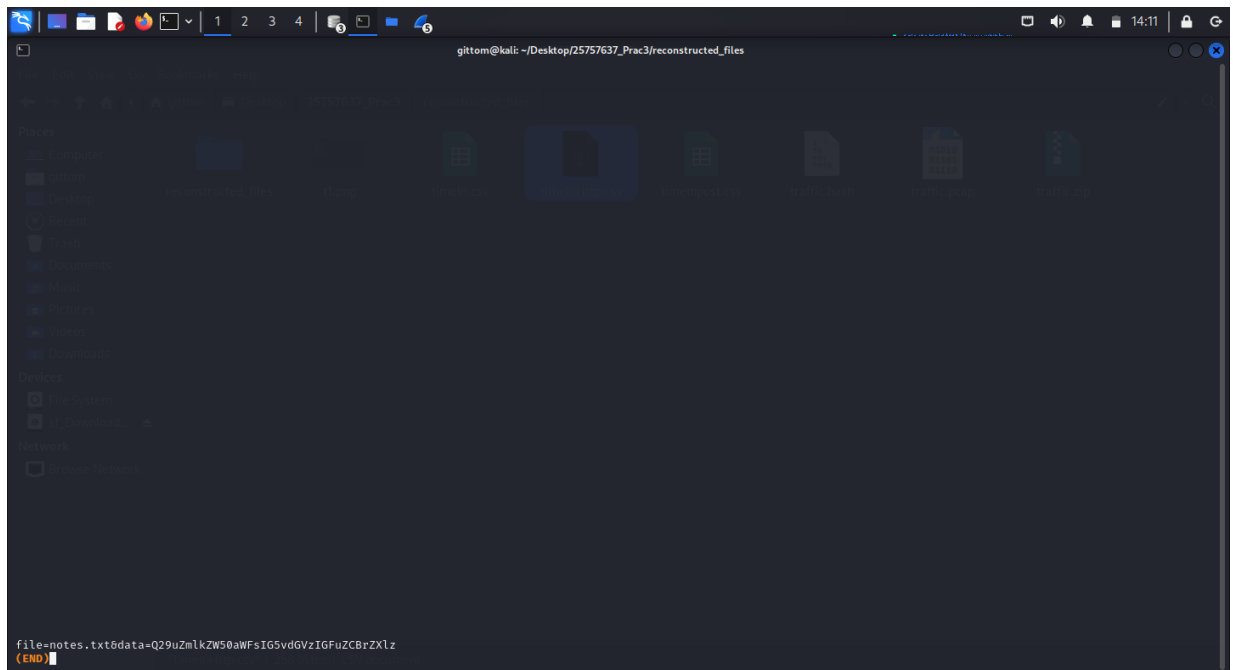


Figure 19: Screenshot of the opened **upload** file

- Type: ASCII text, 56 bytes.
- Decoded Content: Base64 string decoded to “Confidential notes and keys”.
- Hash: Verified using **sha256sum**.
- Bulk Extractor Result: No email, URL, or encoded artifacts detected.

payroll.csv file

command to open and read the file

```
cat payroll.csv
```

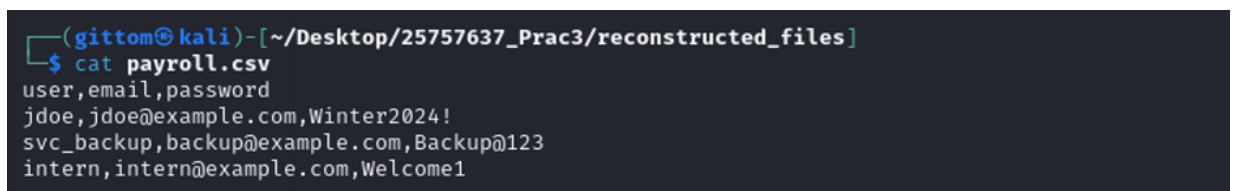


Figure 20: Screenshot of opened **payroll.csv** file

- Type: CSV file, 130 bytes.
- Content: Contains usernames, email addresses, and plaintext passwords in the listed order.
- Emails Found: jdoe@example.com, backup@example.com, intern@example.com.

3. Malware Scanning Tool Attempted: clamav and freshclam

Result: Packages unavailable via Kali repositories.

Alternative Used:

- Bulk Extractor for local pattern-based scanning.
- Manual inspection using `file`, `strings`, and hash reputation lookup.

Conclusion: No malware signatures detected in either file.

4. Timeline of Attack Events

Time	Source	Destination	Protocol	Info
0.000000	192.168.1.10	203.0.113.50	HTTP	POST /upload HTTP/1.1 (application/x-www-form-urlencoded)
0.300000	203.0.113.50	192.168.1.10	HTTP	HTTP/1.1 200 OK
0.600000	192.168.1.10	203.0.113.50	HTTP	GET /files/payroll.csv HTTP/1.1
0.900000	203.0.113.50	192.168.1.10	HTTP	HTTP/1.1 200 OK (text/csv)
1.200000	192.168.1.10	203.0.113.50	FTP	Request: USER jdoe
1.400000	192.168.1.10	203.0.113.50	TCP	[TCP Retransmission] 49154 → 21 [PSH, ACK] Seq=2 Ack=1 Win=64240 Len=18
1.600000	192.168.1.10	198.51.100.23	ICMP	Echo (ping) request id=0x0042, seq=1/256, ttl=64 (reply in 8)
1.800000	198.51.100.23	192.168.1.10	ICMP	Echo (ping) reply id=0x0042, seq=1/256, ttl=64 (request in 7)

Figure 21: Screenshot of the events

Timestamp	Event Type	Description
0.000000	HTTP POST Request	Host 192.168.1.10 sends POST data to 203.0.113.50 using <code>application/x-www-form-urlencoded</code> . Possible data exfiltration.
0.300000	HTTP Response	Server 203.0.113.50 replies with HTTP/1.1 200 OK. Indicates successful receipt of POST data.
0.600000	HTTP GET Request	Host 192.168.1.10 requests <code>/files/payroll.csv</code> from 203.0.113.50. Suggests file retrieval.
0.900000	HTTP Response	Server 203.0.113.50 responds with HTTP/1.1 200 OK, content type <code>text/csv</code> . File download confirmed.
1.200000	FTP Credential Leak	FTP login attempt from 192.168.1.10 to 203.0.113.50 using <code>USER jdoe</code> . Indicates credential exposure.
1.400000	TCP Retransmission	TCP packet retransmitted from 192.168.1.10 to 203.0.113.50. May indicate network instability or stealth technique.
1.600000	ICMP Echo Request	Host 192.168.1.10 sends ICMP ping to 198.51.100.23. Identifier: <code>0x42ed</code> , Sequence: 1/256, TTL: 64.
1.800000	ICMP Echo Reply	Host 198.51.100.23 replies to ping. TTL: 64, Reply Time: 8 ms. Confirms connectivity.

Table 2: Reconstructed Timeline of Malicious events

TCP retransmissions and ICMP traffic are not always harmful, but in this case they might be linked to attacker activity. Retransmissions could mean the attacker is trying to avoid detection or the network is unstable. ICMP pings to outside servers might be used to check if the system is still reachable or to quietly signal control. These events are marked as suspicious and should be considered with the confirmed attacks.

Task 4: Recommendations

1. **Replace insecure protocols and strengthen authentication**

FTP was used to transmit credentials in plaintext, which is highly vulnerable to interception. Replace FTP with secure alternatives like SFTP or FTPS. Enforce strong password policies and multi-factor authentication for all remote access services. Monitor login attempts and alert on unusual patterns, such as repeated access from unexpected IPs.

2. **Monitor and control outbound data transfers**

The HTTP POST to an external IP suggests possible data exfiltration. Deploy outbound traffic monitoring tools that inspect payloads for sensitive content. Use data loss prevention (DLP) systems to block unauthorized uploads and flag suspicious POST requests, especially those using encoded or form-based payloads.

3. **Restrict ICMP and apply network segmentation**

ICMP echo traffic to an external IP may indicate beaconing or connectivity testing by an attacker. Limit ICMP traffic to internal diagnostics only and block outbound pings to unknown destinations. Segment the network to isolate sensitive systems from general user access, reducing the risk of lateral movement.

4. **Detect anomalies through logging and alerting**

TCP retransmissions may be benign, but in forensic contexts they can signal stealth techniques. Implement logging for all network activity and configure alerts for anomalies such as repeated retransmissions, unexpected protocol usage, or access to sensitive files. Use SIEM tools to correlate events and detect patterns that suggest evasion or probing.

5. **Strengthen user awareness and conduct regular audits**

Human error remains a major attack vector. Provide regular training on secure data handling, phishing awareness, and credential hygiene. Conduct periodic forensic audits of network traffic to validate controls and uncover hidden threats. Ensure that all systems are patched and that unused services (like FTP) are disabled by default.